# Asset Processing

@andriyDev

# The default asset workflow

Your project directory:

```
- assets/
  - grass.bmp
- src/
  - main.rs
- Cargo.toml
```

```
fn setup(
  mut commands: Commands,
  asset_server: Res<AssetServer>,
) {
  let handle: Handle<Image> = asset_server.load("grass.bmp");
  commands.spawn(Sprite {
    image: handle,
    ..Default::default()
  });
}
```

# Where did you get your assets?

- I made them!
  - GLTF, PNG, RON, WAV
- Your artist
  - FBX, STL, TGA, MP3
- Purchased an asset pack

Your assets folder is a nightmare.

# Other reasons you might not want to ship your assets folder

- Too large
  - Use compression
  - Binary encoding
- Slow to load at runtime
  - Use a more easy to load format (e.g., KTX2)
- Derived data
  - Derive it at build time and ship the output

# Finally: Asset Processing

Raw source assets -> game-ready assets

# Turning it on

```
fn main() {
  App::new()
    .add_plugins(DefaultPlugins.set(AssetPlugin {
      mode: AssetMode::Processed,
      ..Default::default()
    }))
    .run();
}
```

During dev, enable the `asset_processor` feature (and the `file_watcher` feature).

Now you have an `imported_assets/` directory! Ship this to users!

# The Process trait

```rust
pub trait Process:
    Sized
    + TypePath
    + Send
    + Sync
    + 'static {
    type Settings: Settings + Default + Serialize + for<'a> Deserialize<'a>;
    type OutputLoader: AssetLoader;

    fn process(
        &self,
        context: &mut ProcessContext<'_>,
        settings: &Self::Settings,
        writer: &mut (dyn AsyncWrite + Unpin + Sync + Send + 'static),
    ) -> impl ConditionalSendFuture<
        Output = Result<<Self::OutputLoader as AssetLoader>::Settings, ProcessError>,
    >;
}
```

# The Process trait (simplified)

```rust
pub trait Process: TypePath {
    type Settings: Settings + Default + Serialize + for<'a> Deserialize<'a>;
    type OutputLoader: AssetLoader;

    async fn process(
        &self,
        context: &mut ProcessContext<'_>,
        settings: &Self::Settings,
        writer: &mut (dyn AsyncWrite + Unpin + Sync + Send + 'static),
    ) -> Result<<Self::OutputLoader as AssetLoader>::Settings, ProcessError>;
}
```

TL;DR: read bytes from the `ProcessContext`, write bytes to `writer`.

# LoadTransformAndSave

3 generic arguments:

- `AssetLoader` type: bytes -> T
- `AssetTransformer` type: T -> U
- `AssetSaver` type: U -> bytes

`IdentityAssetTransformer` is an easy way to fill in that middle type.

# Using your processor

```
use RonChangeProcessor = LoadTransformAndSave<
  RonLoader,
  DoSomething,
  RonSaver,
>;
let saver = RonSaver::from_world(app.world_mut());
app
  .register_asset_processor(
    RonChangeProcessor::new(DoSomething, saver),
  )
  .set_default_asset_processor::<RonChangeProcessor>(
    "cool.ron",
  );
```

Great for general-purpose processing (e.g., texture compression).

# Using your processor (part 2)

```
// Imagine we enable the `compressed_image_saver` feature.
fn setup(
  mut commands: Commands,
  asset_server: Res<AssetServer>,
) {
  let handle: Handle<Image> = asset_server.load("grass.bmp");
  commands.spawn(Sprite {
    image: handle,
    ..Default::default()
  });
}
```

# Meta files

To create a meta file for `my_docs.cool.ron`, create a file like this at `my_docs.cool.ron.meta`.

```
(
    meta_format_version: "1.0",
    asset: Process(
        processor: "LoadTransformAndSave<RonLoader, DoSomething,
        settings: (
            loader_settings: (),
            transformer_settings: (),
            saver_settings: (),
        ),
    ),
)
```

Aside: Meta files can also be used to change how loading happens.

# Processing is fairly flexible

- `ProcessContext::asset_reader` gives you access to the raw bytes.
  - You aren't restricted to using a loader!
- `ProcessContext::load_source_asset` allows us to use asset loaders to load our asset.
  - You can make loaders *just* for processing.
  - You can make asset types *just* for processing.
- (the API could be nicer tho)

# Examples

- Make a `RawGltf` and `RawGltfLoader`.
  - Might be easier to extract the data you need rather than going through the `Gltf` asset with its Scenes.
- Unprocessed "list of level file paths" processed into "list of level metadata"
  - Make a "fake" asset loader that nested-loads the level file paths, computes the metadata, and then save the asset as RON.

# Gaps

- One-to-many processing
- Per-platform processing
- Standalone processing
- Manual processing
- Folder-level meta files
- Publish workflow

# Thanks for listening!

Come chat with us in #assets-dev so we can better support your uses of asset processing!